

GOLD-001 — The Complete Record

Production RAG v1 · six closed batches · 150 holdout-eligible cases · corpus snapshot
snap_689e336380a054d8039dc35b2c09cd0a · status generated 2026-08-31T03:37:38Z

WHERE THE PROJECT STANDS

150 human-verified cases, all 150 of them holdout-eligible, drawn from 159 candidates reviewed. 9 were rejected and every one is kept as a negative audit example. They arrive in two groups: six closed generated batches (90 cases) and one owner-reviewed packet of 60 standalone drafts (HA-01...HA-60).

No retrieval system has ever been run against a GOLD candidate. No holdout and no validation split is frozen. SYSTEM-A and SYSTEM-B remain frozen and unexecuted.

READ THIS BEFORE QUOTING THE NUMBER

150 cases is the achieved GOLD benchmark-size target. It is a size, not coverage.

96 of 150 eligible cases come from one provider (openai). A per-provider result from this set is not a comparison between providers.

the largest recorded category holds 58 of 150 cases and the smallest holds 1; 33 cases predate the field entirely. An unweighted score over this set is dominated by the largest category.

Genuine multi-hop: 1 case (GOLD-B004-15). 1 case. One observation cannot support any claim about multi-hop performance, and 150 does not change that.

The sixty HA cases were authored and reviewed **outside** the preregistered sequence, and the calibration pilot that was supposed to precede them **was never run**. That is recorded as `ACCEPTED_PROTOCOL_DEVIATION` and is set out in section 11.

Corpus reproduction is **incomplete; 139 Anthropic documents and 2482 unbuildable identities outstanding**, and retrieval is `RETRIEVAL_BLOCKED` .

150
human-verified

150
holdout-eligible

9
rejected, all preserved

96 / 54
openai / anthropic —
not a balanced set

1. The six batches and the HA group

2. What is in the benchmark

3. Every approved case

4. Every rejection

5. Batch 004 — multi-hop, measured

6. Batch 005 — two lanes

7. Batch 006 — the four fixes, and the shortfall

8. The generator defect ledger

9. Modules built

10. Heading parser audit

11. The HA packet, and the protocol deviation

11b. Batch 007 — fixes shipped, pilot blocked

12. Tests and invariants

13. Commits and artifacts

1. The six batches and the HA group

A batch closes only when every candidate has a final human decision. The closure records a hash over its candidate records, and the test suite re-checks that hash, so an edit after closure fails the tests rather than passing unnoticed.

The HA row is not a batch. Those sixty cases were authored as standalone drafts and admitted from an owner-reviewed packet, not generated by the pipeline the six batches ran through. Its 100% acceptance is not comparable to a batch's: a generated batch is reviewed after generation, while this packet was assembled and reviewed together. Section 11 sets out what that does and does not establish.

group	origin	closed	candidates	verified	rejected	acceptance	eligible	closure hash
001	generated batch	2026-08-20	18	16	2	89%	16	d6f92e8d1a...
002	generated batch	2026-08-20	18	17	1	94%	17	69364f672e...
003	generated batch	2026-08-21	20	20	0	100%	20	e186a2f30e...
004	generated batch	2026-08-23	15	14	1	93%	14	57269018d2...
005	generated batch	2026-08-24	19	15	4	79%	15	ffbf9dda40...
006	generated batch	2026-08-25	9	8	1	89%	8	7ff5596c75...
HA	owner-reviewed packet	2026-08-31	60	60	0	100%	60	32b59f774d...
all			159	150	9	94%	150	

THREE STATES, DELIBERATELY KEPT APART

`precheck_holdout_ready` is **structural only** — a script's verdict that a record is checkable. `human_verified` is an **owner decision** and nothing else can set it. `holdout_eligible` is **derived**: a deterministic gate re-run against the current records at every closure.

The distinction is load-bearing. Batch 005 shipped 19 of 19 precheck-ready and its review still repaired seven and rejected four. Batch 006 shipped 9 of 9 precheck-ready and the owner still rejected one and revised five.

2. What is in the benchmark

59

distinct source documents

54 / 96

anthropic / openai

126 / 24

single-span / multi-span

reasoning type	cases	share
<code>exact_lookup</code>	58	39%
<code>configuration_interaction</code>	23	15%
<code>error_behavior</code>	19	13%
<code>lifecycle_compatibility_migration</code>	8	5%
<code>request_response</code>	5	3%
<code>lifecycle</code>	2	1%
<code>ambiguity_disambiguation</code>	1	1%
<code>genuine_multi_hop</code>	1	1%

Every case is anchored to exact character offsets in the frozen corpus, carries an evidence hash, and lists the literal critical strings its claims depend on. Ground truth never depends on chunk IDs, so a chunking change cannot silently invalidate the benchmark.

3. Every approved case

All 150 human-verified cases, in id order. Batch 001's rows come from its v2 overlay, which supersedes the closed v1 record for eligibility.

id	prov	reasoning type	question
GOLD-B001-01	anth	☐	
GOLD-B001-02	open	☐	
GOLD-B001-03	anth	☐	
GOLD-B001-04	open	☐	
GOLD-B001-05	anth	☐	
GOLD-B001-06	open	☐	
GOLD-B001-07	anth	☐	
GOLD-B001-08	open	☐	
GOLD-B001-09	anth	☐	
GOLD-B001-10	open	☐	
GOLD-B001-11	anth	☐	
GOLD-B001-12	open	☐	
GOLD-B001-13	anth	☐	
GOLD-B001-14	anth	☐	
GOLD-B001-17	anth	☐	
GOLD-B001-18	anth	☐	
GOLD-B002-01	anth	☐	
GOLD-B002-02	anth	☐	
GOLD-B002-03	open	☐	
GOLD-B002-04	anth	☐	
GOLD-B002-05	anth	☐	
GOLD-B002-06	anth	☐	
GOLD-B002-07	anth	☐	
GOLD-B002-08	anth	☐	
GOLD-B002-09	anth	☐	
GOLD-B002-10	anth	☐	
GOLD-B002-11	anth	☐	
GOLD-B002-13	anth	☐	
GOLD-B002-14	anth	☐	
GOLD-B002-15	open	☐	

id	prov	reasoning type	question
GOLD-B002-16	anth		
GOLD-B002-17	open		
GOLD-B002-18	anth		
GOLD-B003-01	anth	configuration_interaction	
GOLD-B003-02	anth	configuration_interaction	
GOLD-B003-03	anth	error_behavior	
GOLD-B003-04	anth	error_behavior	
GOLD-B003-05	anth	error_behavior	
GOLD-B003-06	anth	error_behavior	
GOLD-B003-07	anth	exact_lookup	
GOLD-B003-08	anth	exact_lookup	
GOLD-B003-09	anth	exact_lookup	
GOLD-B003-10	anth	lifecycle	
GOLD-B003-11	anth	lifecycle	
GOLD-B003-12	anth	exact_lookup	
GOLD-B003-13	open	configuration_interaction	
GOLD-B003-14	open	configuration_interaction	
GOLD-B003-15	open	exact_lookup	
GOLD-B003-16	open	exact_lookup	
GOLD-B003-17	open	exact_lookup	
GOLD-B003-18	open	exact_lookup	
GOLD-B003-19	open	exact_lookup	
GOLD-B003-20	open	exact_lookup	
GOLD-B004-01	anth	error_behavior	When using server tools, what does the API return if the server-side sampling loop reaches its i
GOLD-B004-02	anth	configuration_interaction	What happens if Claude calls web search and one of your client tools in the same group of parall
GOLD-B004-03	anth	error_behavior	What happens if the most recent assistant message contains <code>thinking</code> or <code>redacted_thinking</code> blo
GOLD-B004-04	anth	error_behavior	On Claude 4.5 models and newer, if input tokens plus <code>max_tokens</code> exceed the context window size
GOLD-B004-05	anth	error_behavior	What happens if Claude attempts more searches than allowed?
GOLD-B004-06	anth	exact_lookup	Within the <code>user_location</code> parameter, what value does the <code>timezone</code> field take?
GOLD-B004-07	anth	lifecycle_compatibility_mi	If I need a hard ceiling on thinking costs, what is the support status of extended thinking with

id	prov	reasoning type	question
GOLD-B004-09	open	ambiguity_disambiguation	In a <code>FunctionToolCallArgumentsDeltaEvent</code> , what does the <code>parsed_arguments</code> field contain, and
GOLD-B004-10	open	configuration_interaction	<code>RealtimeRunner</code> uses <code>OpenAIRealtimeWebSocketModel</code> by default. What happens if I pass a differ
GOLD-B004-11	open	error_behavior	In a streamed run, what happens if a tool requires approval?
GOLD-B004-12	open	configuration_interaction	What happens if you are manually continuing from <code>result.to_input_list(mode="normalized")</code> , and
GOLD-B004-13	open	exact_lookup	For a handoff, what does the <code>input_type</code> option specify?
GOLD-B004-14	open	exact_lookup	In the <code>handoff()</code> function, what does the <code>input_filter</code> option do?
GOLD-B004-15	open	genuine_multi_hop	If I set <code>needs_approval</code> to <code>True</code> on a function tool used with hosted agents, what happens?
GOLD-B005-02	anth	configuration_interaction	What happens when <code>jwt.type</code> is <code>discovery</code> and no <code>discovery_base</code> is set?
GOLD-B005-03	anth	error_behavior	What happens when filtering by a non-existent <code>deployment_id</code> ?
GOLD-B005-04	anth	error_behavior	What happens if <code>encrypted_content</code> is missing or modified?
GOLD-B005-05	anth	exact_lookup	What constraint does a request-level <code>allowed_domains</code> list have to satisfy?
GOLD-B005-07	anth	lifecycle_compatibility_mi	Is <code>compaction_control</code> still supported?
GOLD-B005-08	anth	lifecycle_compatibility_mi	Is manual extended thinking with <code>budget_tokens</code> supported on Claude Sonnet 5?
GOLD-B005-09	open	error_behavior	What happens if the arguments are malformed JSON, are valid JSON but not an object (for example,
GOLD-B005-11	open	configuration_interaction	When using the <code>bedrock(...)</code> provider, what does setting <code>AWS_BEDROCK_BASE_URL</code> change?
GOLD-B005-12	open	configuration_interaction	What does <code>FuseMountPattern</code> require?
GOLD-B005-14	open	configuration_interaction	What does <code>ApplyPatchTool</code> require?
GOLD-B005-15	open	configuration_interaction	When a <code>ComputerTool</code> is present, how are <code>tool_choice="computer"</code> , <code>"computer_use"</code> and <code>"compu</code>
GOLD-B005-16	open	configuration_interaction	When you use a <code>Session</code> such as <code>SQLiteSession</code> , what usage does each call to <code>Runner.run(...)</code>
GOLD-B005-17	open	error_behavior	What happens if we exceed the <code>max_turns</code> passed?
GOLD-B005-18	open	exact_lookup	What must an Undici-specific option like <code>dispatcher</code> be paired with?
GOLD-B005-19	open	lifecycle_compatibility_mi	What happened to <code>httpAgent</code> ?
GOLD-B006-01	anth	exact_lookup	What credential is required to create an <code>admin</code> -role service account, and what role may a workl
GOLD-B006-02	anth	error_behavior	What happens if <code>role: "system"</code> is used in <code>messages</code> with Claude Opus 4.7?
GOLD-B006-03	anth	lifecycle_compatibility_mi	How do the newer code-execution tool versions behave on Claude Haiku 4.5?
GOLD-B006-04	anth	exact_lookup	What effort level does Claude Sonnet 5 default to on the Claude API and Claude Code?

id	prov	reasoning type	question
GOLD-B006-05	anth	exact_lookup	What does <code>thinking.display</code> default to on <code>claude-mythos-5</code> and <code>claude-fable-5</code> ?
GOLD-B006-07	open	configuration_interaction	What does Setting <code>audio.input.turn_detection</code> to <code>None</code> disable?
GOLD-B006-08	open	lifecycle_compatibility_mi	What does the OpenAI Python SDK's temporary legacy-client compatibility path require, and how sh
GOLD-B006-09	open	exact_lookup	What does <code>RunErrorHandlerResult.include_in_history</code> default to?
HA-01	open	configuration_interaction	In the OpenAI Agents SDK for Python, when a session implementation exposes default session setti
HA-02	open	lifecycle_compatibility_mi	Does the OpenAI Agents SDK for Python convert arbitrary legacy <code>httpx</code> objects to HTTPX2?
HA-03	open	configuration_interaction	In the OpenAI Agents SDK for Python, when a harness ID is configured, what stops the SDK adding
HA-04	open	configuration_interaction	In the OpenAI Agents SDK for Python, why do output guardrails not support the <code>run_in_parallel</code>
HA-05	open	error_behavior	In the OpenAI Agents SDK for Python, when a model-call attempt exceeds the <code>ModelSettings.timeou</code>
HA-06	open	lifecycle_compatibility_mi	In the OpenAI Agents SDK for Python with MCP Python SDK v2 installed, what does the <code>mcp.Client</code>
HA-07	open	exact_lookup	In the OpenAI Agents SDK for Python, what is the default for <code>max_size</code> in the <code>transport_config</code>
HA-08	open	error_behavior	In the OpenAI Agents SDK for Python, what happens if a dataclass configuration type defined by t
HA-09	open	configuration_interaction	In the OpenAI Agents SDK for Python, which environment variable supplies the websocket <code>/respons</code>
HA-10	open	configuration_interaction	In the OpenAI Agents SDK for Python, what do you pass when setting the default key or client so
HA-11	open	error_behavior	In the OpenAI Agents SDK for Python, which exceptions do tool guardrails use for tripwires?
HA-12	open	error_behavior	In the OpenAI Agents SDK for Python, for an agent-level tripwire, which attribute of the excepti
HA-13	open	exact_lookup	In the OpenAI Agents SDK for Python, what does <code>exception.run_data.input_guardrail_results</code> cont
HA-14	open	error_behavior	In the OpenAI Agents SDK for Python, do runner-managed failures such as <code>MaxTurnsExceeded</code> prese
HA-15	open	request_response	In the OpenAI Agents SDK for Python, where is the JSON returned for a handoff tool call validate
HA-16	open	configuration_interaction	In the OpenAI Agents SDK for Python, when <code>RunConfig.nest_handoff_history</code> is enabled, what abse
HA-17	open	request_response	In the OpenAI Agents SDK for Python, where does an interruption surface when it comes from a nes
HA-18	open	configuration_interaction	In the OpenAI Agents SDK for Python, what is the scope of a per-call approval?
HA-19	open	configuration_interaction	In the OpenAI Agents SDK for Python, do sticky approval decisions survive serializing and restor

id	prov	reasoning type	question
HA-20	open	configuration_interaction	In the OpenAI Agents SDK for Python, does an always-approve decision for a hosted MCP tool carry
HA-21	open	request_response	In the OpenAI Agents SDK for Python, what happens if you rerun after resolving only some interrupt
HA-22	open	exact_lookup	In the OpenAI Agents SDK for Python, if you are also using a session, what session should you pass
HA-23	open	request_response	In the OpenAI Agents SDK for Python, with SDK-default nested handoff history enabled, how do sessions
HA-24	open	request_response	In the OpenAI Agents SDK for Python, under what condition can <code>ToolContext</code> also expose <code>.tool_info</code>
HA-25	open	exact_lookup	In the OpenAI Agents SDK for Python, when is the default <code>OPENAI_API_KEY</code> resolved?
HA-26	open	exact_lookup	In the OpenAI Agents SDK for Python, what client instance does the SDK create by default?
HA-27	open	exact_lookup	In the OpenAI Agents SDK for Python, what <code>openai</code> version range does SDK version 0.21.0 require
HA-28	open	exact_lookup	In the OpenAI Agents SDK for Python, which HTTP dependency is used for local MCP transports by default
HA-29	open	exact_lookup	In the OpenAI Agents SDK for Python, which OpenAI API is used by default?
HA-30	open	exact_lookup	In the OpenAI Agents SDK for Python, if no SDK default harness ID is set, which environment variable
HA-31	open	exact_lookup	In the OpenAI Agents SDK for Python, how can tracing remain enabled while potentially sensitive
HA-32	open	exact_lookup	In the OpenAI Agents SDK for Python, does the SDK attach handlers by default to <code>openai.agents</code>
HA-33	open	exact_lookup	In the OpenAI Agents SDK for Python, with tool-data redaction enabled, what diagnostic error is
HA-34	open	exact_lookup	In the OpenAI Agents SDK for Python, do nested <code>Agent.as_tool()</code> runs receive an isolated copy of
HA-35	open	exact_lookup	In the OpenAI Agents SDK for Python, is the local context object sent to the LLM?
HA-36	open	exact_lookup	In the OpenAI Agents SDK for Python, what context-type constraint applies to agents, tool functions
HA-37	open	exact_lookup	In the OpenAI Agents SDK for Python, what does <code>RunContextWrapper.usage</code> expose?
HA-38	open	exact_lookup	In the OpenAI Agents SDK for Python, which <code>ToolContext</code> field contains the raw argument string
HA-39	open	exact_lookup	In the OpenAI Agents SDK for Python, what does <code>ToolContext.qualified_tool_name</code> contain when a
HA-40	open	exact_lookup	In the OpenAI Agents SDK for Python, what value of <code>transport_config.ping_interval</code> disables CLI
HA-41	open	exact_lookup	In the OpenAI Agents SDK for Python, what does setting <code>transport_config.ping_timeout=None</code> permit

id	prov	reasoning type	question
HA-42	open	exact_lookup	In the OpenAI Agents SDK for Python, what wait does <code>transport_config.handshake_timeout</code> bound f
HA-43	open	exact_lookup	In the OpenAI Agents SDK for Python, what is <code>RealtimeModelConfig.playback_tracker</code> used to repo
HA-44	open	exact_lookup	In the OpenAI Agents SDK for Python, which filter takes precedence when both a handoff's <code>input_</code>
HA-45	open	exact_lookup	In the OpenAI Agents SDK for Python, is the opt-in beta nested handoff history enabled by default
HA-46	open	exact_lookup	In the OpenAI Agents SDK for Python, which function restores the default generated handoff-histo
HA-47	open	exact_lookup	In the OpenAI Agents SDK for Python, does a handoff's <code>input_type</code> replace the next agent's main
HA-48	open	exact_lookup	In the OpenAI Agents SDK for Python, for a function tool requiring approval, does passing pre-ap
HA-49	open	exact_lookup	In the OpenAI Agents SDK for Python, do function-tool guardrails apply to the handoff call itself
HA-50	open	exact_lookup	In the OpenAI Agents SDK for Python, which identity fields must be non-empty for a hosted MCP al
HA-51	open	exact_lookup	In the OpenAI Agents SDK for Python, does using <code>context_override</code> when loading <code>RunState</code> remov
HA-52	open	exact_lookup	In the OpenAI Agents SDK for Python, under <code>RunState</code> 's <code>strict_context=True</code> , what allows a non
HA-53	open	exact_lookup	In the OpenAI Agents SDK for Python, does <code>ModelSettings.timeout</code> bound the full agent run, func
HA-54	open	exact_lookup	In the OpenAI Agents SDK for Python, what must be configured for the SDK to retry general model
HA-55	open	exact_lookup	In the OpenAI Agents SDK for Python, on Responses websocket transport, will the SDK replay a req
HA-56	open	exact_lookup	In the OpenAI Agents SDK for Python, in <code>ModelRetrySettings</code> , does <code>backoff.max_delay</code> cap expli
HA-57	open	exact_lookup	In the OpenAI Agents SDK for Python, can <code>approve_unsafe_replay=True</code> authorize streamed retries
HA-58	open	exact_lookup	In the OpenAI Agents SDK for Python, for follow-up requests using <code>previous_response_id</code> or con
HA-59	open	exact_lookup	In the OpenAI Agents SDK for Python, when an agent overrides only <code>retry.max_retries</code> , can it re
HA-60	open	exact_lookup	In the OpenAI Agents SDK for Python, which retry settings survive serialized <code>ModelSettings</code> , an

4. Every rejection

All 9 rejected candidates are kept in their batch records as negative audit examples. None was deleted, and none was replaced with a substitute to keep a count up.

id	batch	why
GOLD-B001-15	001	REJECT — code/example false binding. <code>required: ["location"]</code> belongs to the tool input schema in the example request and does not establish a general requirement on <code>tool_choice</code> .
GOLD-B001-16	001	REJECT — concerns the implementation behaviour of a local example helper function rather than an externally meaningful API or documentation fact. Stronger benchmark material is preferred.
GOLD-B002-12	002	REJECT — OVERSIZED_EVIDENCE_ANCHOR / EVIDENCE_PRECISION_COST. The documentation is correct and the fact is real. Making the anchor self-contained required expanding it from ~70 to ~1,430 cha
GOLD-B004-08	004	Rejected: BENCHMARK_QUALITY / TAXONOMY_MISCLASSIFICATION. The facts are not disputed — ContentDeltaEvent carries type "content.delta" and ContentDoneEvent carries type "content.done" — but t
GOLD-B005-01	005	Rejected: BENCHMARK_QUALITY / TAXONOMY_MISCLASSIFICATION / SCOPE_DEFECT. <code>invalid_tool_input</code> is the same semantic concept in both sources — the tool input is invalid — and the examples diff
GOLD-B005-06	005	Rejected: UNANSWERABLE_AS_ASKED / CATEGORY_MISCLASSIFICATION. The evidence is a schema description of what model IDs may appear in <code>fallbacks[i].model</code> and what an empty list means. It does
GOLD-B005-10	005	Rejected: RELATION_DIRECTION / UNRECOVERABLE_SCOPE. The evidence says the experimental model rejects caller-supplied <code>betas</code> overrides; it does not say <code>betas</code> overrides anything, so the gen
GOLD-B005-13	005	Rejected: DUPLICATE_RELATION / BENCHMARK_REDUNDANCY. The fact is supported — <code>S3FilesMountPattern</code> requires broad acknowledgement because <code>mount.s3files</code> uses ambient IAM authority — but it
GOLD-B006-06	006	REJECT — DUPLICATE_FACT / BENCHMARK_REDUNDANCY. The fact is supported, but GOLD-B005-11 already carries substantially the same operational relation from the OpenAI Python library; this obtai

5. Batch 004 — multi-hop, measured rather than assumed

Batch 004 set out to find genuinely composable multi-hop questions: cases where the answer requires combining two spans and neither span alone suffices. It tested **559** identifier-sharing pairs and found **1**.

why a pair was rejected	pairs
span 1 alone answered the full question	16
span 2 alone answered the full question	16
the spans were two unrelated lookups	379
no bridge relationship existed	147
the composed answer introduced unsupported inference	0
rejected	558
passed	1

A near-miss diagnostic re-examined the 5 pairs that failed on a single gate. All were **correct rejections** — the rule under test was *states_dependency* — *span 2's conditional must test the bridge entity's own state*. The best-known of them, `max_tokens`, is a request parameter in one span and a `stop_reason` value in the other: the same token, two different things. That near-miss is why `rag_v1.gold.bridge_equivalence` exists, and it is now a regression test.

Batch 004 closed at 14 of 15 with 6 anchor repairs and 3 owner overrides. An override never deletes a finding: the finding stays on the record next to the owner's decision to accept it.

6. Batch 005 — two lanes, and a second multi-hop measurement

Batch 005 split its effort. Lane A spent it where the corpus pays — interactions, constraints, lifecycle statements, cross-component ambiguity. Lane B searched for chains **dependency-first**, opening only on sentences that state a dependency, rather than batch 004's all-pairs sweep.

dependency-first search	count
dependency pairs considered	3
failed span independence	1
failed semantic equivalence	1
passed	1
budget	1000
new unique chains exported	0

TWO SEARCHES, TWO METHODS, ONE CHAIN

The single valid chain batch 005 found was the chain batch 004 had already closed, so batch 005 exported none. **That is a result about the corpus, not a failed search:** this frozen snapshot contains very little naturally composable multi-hop structure. It is why batch 006 was instructed not to search again, and why the project's multi-hop count is 1 and is not presented as coverage.

Batch 005 targeted 30 and exported 19; of the 46 candidates that reached the semantic self-review, 27 were dropped. It closed at 15 of 19 — the lowest acceptance rate of the six (79%), and the batch that produced four of the seven generator defects in the ledger below.

7. Batch 006 — the four fixes, and a shortfall worth having

Batch 006 implemented the four fixes batch 005's closure preregistered, **before** authoring anything, then targeted 28 candidates and exported 9.

the census behind the shortfall	count
facts mined	1361
distinct evidence spans the miners reach	773
of those, unspent by any closed batch	699
mined facts that reached no builder	2482

THE FINDING THAT CHANGED THE PLAN

The corpus is not exhausted. The authoring is. 699 distinct evidence spans in the frozen snapshot have never been used by any closed batch, and no deterministic template could turn them into a question without paraphrasing — what remains is long, multi-clause prose, and a template that fits it is a template that invents wording.

Refusing all paraphrase is precisely what keeps those facts out of the benchmark. That is the problem batch 007 is preregistered to solve.

The owner approved 8 of 9: five after revisions they specified, three as generated, one rejected. Three taxonomy corrections were applied — a requirement mislabelled as a configuration interaction, a compatibility statement as an exact lookup, a migration note as an interaction — and **no anchor moved**, which a test now enforces. Every revision is attributed to the owner rather than to Claude's review, because a change someone asked for and a change the author proposed are different acts.

The one rejection, and the gate that missed it

`GOLD-B006-06` was rejected as **DUPLICATE_FACT / BENCHMARK_REDUNDANCY**. The fact is supported, but `GOLD-B005-11` already carries the same operational relation from the OpenAI Python library while this obtained it from the TypeScript library. Duplicate control compares question text, span offsets and span text — and two libraries documenting the same behaviour share none of those. Recorded as defect E.

8. The generator defect ledger

Every defect a review found in the generator, what case exposed it, and where it was or will be fixed. A defect is **recorded, never patched into a closed batch**: the generation artifact stays as generated, and the fix belongs in the next batch's preregistration where it can be declared before it sees a candidate.

defect	seen in	found	fixed in
A the bare-definition-bullet rule only inspects single-span records	<code>GOLD-B005-01</code>	batch 005 review	batch 006
B markdown reference links survive into questions built from conditional sentences	<code>GOLD-B005-15</code>	batch 005 review	batch 006
C prose mistaken for a section heading by the parser	<code>GOLD-B005-11</code>	batch 005 review	batch 006
D subject–relation direction must remain explicitly checked	<code>GOLD-B005-10</code>	batch 005 review	batch 006
E cross-library duplicate facts are invisible to duplicate control	<code>GOLD-B006-06</code>	batch 006 review	batch 007 (preregistered)

	defect	seen in	found	fixed in
F	compound single-span facts are labelled by their first verb	GOLD-B006-01, GOLD-B006-03, GOLD-B006-08	batch 006 review	batch 007 (preregistered)
G	questions inherit the breadth of their frame, not of their evidence	GOLD-B006-02, GOLD-B006-04, GOLD-B006-05	batch 006 review	batch 007 (preregistered)

A. the bare-definition-bullet rule only inspects single-span records

The generation self-review drops a candidate whose single span is a bare '`field`: description' bullet, because its scope lives in the heading. B005-01 is two such bullets in a multi-span record, and the rule's `len(spans) == 1` guard let it through. Recorded rather than patched: batch 005's generation artifact is not being regenerated, and a fix belongs in batch 006's preregistration.

Fix: Extend the bare-definition-bullet scope rule to every span of a multi-span record, not only records with exactly one span.

B. markdown reference links survive into questions built from conditional sentences

The link stripper runs on the composed question, but a reference-style link whose label is itself backticked was not matched. Repaired here by hand; the pattern should be fixed before batch 006.

Fix: Strip markdown reference links whose label is itself backticked, before a question is composed.

C. prose mistaken for a section heading by the parser

`section_path` reads 'configured through AWS_REGION, AWS_DEFAULT_REGION, or your AWS profile.' No claim depends on it, and closed batches must not be touched, so this is recorded as a parser observation for a future audit.

Fix: Audit the heading parser against the corpus snapshot and record how often ordinary prose is captured as a `section_path`. Do NOT fix historical parser output in place; closed batches keep what they have.

D. subject-relation direction must remain explicitly checked

Rejected: `RELATION_DIRECTION / UNRECOVERABLE_SCOPE`. The evidence says the experimental model rejects caller-supplied `betas` overrides; it does not say `betas` overrides anything, so the generated question reverses the documented relation. Rewriting around the true subject would require identifying the experimental model, whose identity is outside the exact evidence and cannot be recovered by a minimal valid expansion. Not salvaged.

Fix: Every generated question must be re-read against its span for relation direction: the question's subject must be the source's subject, not its object.

E. cross-library duplicate facts are invisible to duplicate control

Duplicate control compares normalised question text, span offsets and span text. Two provider libraries documenting the same operational behaviour share none of those, so the same fact can enter the benchmark twice from two SDKs. GOLD-B005-11 and GOLD-B006-06 both state that a base-URL environment variable overrides the region-derived Bedrock endpoint.

Fix: Compare candidates on their (subject, relation, object) triple, normalised, in addition to text and offsets. Batch 006 records that triple on every candidate, so the material for the check now exists. Flag rather than auto-drop: two libraries genuinely differing in behaviour is a real case, and only a reviewer can tell the two apart.

F. compound single-span facts are labelled by their first verb

The predicate lane picks a frame from the first matching verb and takes the reasoning type from that frame. Three of the nine exported candidates were relabelled by the owner: a requirement read as a configuration interaction, a compatibility statement read as an exact lookup, and a migration note read as a configuration interaction. The evidence was right in every case; the taxonomy was not.

Fix: Classify from the whole sentence rather than from the matched verb: a span naming a support status, a version or a migration is a lifecycle case whatever its verb, and `configuration_interaction` should require two settings that bear on each other rather than one requirement with two identifiers in it.

G. questions inherit the breadth of their frame, not of their evidence

'What does X reject?' and 'What does X default to?' ask for a complete list. The evidence gives one item, usually scoped to named models or surfaces. Three candidates needed rescoping, and in two of them the scope qualifier was not even in the critical strings, so nothing checked it.

Fix: When the source sentence carries a model, platform or surface qualifier, the question must carry it too and it must appear in the critical strings. Add a generation gate: a question whose evidence is scoped and whose wording is not, does not export.

9. Modules built

Each exists because a specific candidate got through a gate that should have caught it. Every one carries a regression test built from that candidate, not from an invented example — a test written from an imagined case tests the rule the author had in mind, and these rules exist because the author's rule missed the real thing.

module	what it checks	added	because of
<code>rag_v1.gold.scoping</code>	bare definition bullets, asked of every span	batch 006	GOLD-B005-01
<code>rag_v1.gold.relations</code>	source and question triples; direction check	batch 006	GOLD-B005-10
<code>rag_v1.gold.questionform</code>	question form must match evidence form	batch 006	GOLD-B005-08, -18
<code>rag_v1.gold.normalisation</code>	markdown link stripping (3 link shapes)	batch 006	GOLD-B005-15
<code>rag_v1.gold.defects</code>	one defect shape across six renderers	batch 006	three renderers crashed identically
<code>rag_v1.gold.bridge_equivalence</code>	a bridge entity must mean the same thing in both spans	batch 005	max_tokens near-miss
<code>rag_v1.gold.mining_v5</code>	interaction, constraint, lifecycle and ambiguity miners	batch 005	question subject ≠ fact subject
<code>rag_v1.gold.multihop</code>	dependency-first chain search	batch 005	batch 004's 559-pair all-pairs search
<code>rag_v1.gold.authoring</code>	question builders, including the batch-006 predicate lane	batches 005-006	2482 facts no builder could reach
<code>rag_v1.gold.eligibility</code>	the deterministic holdout gate (6 conditions)	batch 005	required_evidence_declared added

10. Heading parser audit

44 of 5857 parsed headings (0.75%) read as ordinary prose rather than as a label, across 15 of 202 documents. 82 were suspicious on at least one rule.

why a heading was flagged	headings
starts lowercase	30
ends in a sentence-final period	21
contains a comma-separated 'or'/'and' list	16
longer than 12 words	14
ends in a comma, semicolon or colon	13

44 of 5857 parsed headings (0.75%), in 15 of 202 documents. That is isolated. It does not justify a parser experiment on its own, and the batch-006 rule — never trust `section_path` for scope — is the cheaper fix.

Nothing was rewritten. No heading changed, no document was reparsed into storage, no evidence anchor moved. A closed case approved against a bad `section_path` was approved against its *evidence*; the path is metadata beside it. What changed is a rule: `section_path` is never trusted for claim scope, and a candidate's exact evidence must carry the scope its claim needs.

11. The HA packet, and the protocol deviation

Sixty standalone drafts, `HA-01` through `HA-60`, were authored outside the preregistered sequence and admitted from an owner-reviewed packet (`Production_RAG_v1_Full_150_Case_Review.pdf`, sha256 `bf6190fc53ee4ada...`). All sixty are OpenAI-sourced. They are what carried the project from 90 to 150.

DISPOSITION: ACCEPTED_PROTOCOL_DEVIATION

The batch-007 preregistration required a prospective 10-case calibration pilot, run and independently reviewed, **before** evidence-grounded paraphrasing was scaled. **The pilot was never run.** The sixty cases were authored first.

These sixty cases are not the preregistered pilot and may not be described as it, retrospectively or otherwise. The pilot remains unrun.

A prospective pilot is a commitment made before the data is seen; its value is that the thresholds cannot be chosen to fit the result. Reviewing sixty cases afterwards, however carefully, cannot recover that property. What the downstream checks establish is that these particular cases are sound. What they do not establish is that the method was calibrated before it was used.

Two case-level records from the admission are worth naming because they are the kind of thing that usually disappears into a total:

- `HA-15` carries a retained anaphora finding — *refers to 'the model' with no antecedent in the span* — accepted by the owner as `NONCRITICAL_ANAPHORA`. The override did not delete the finding; it sits on the record beside the decision.
- `HA-47` had its two spans merged into one contiguous anchor (608 characters). The prior and new hashes are both recorded, so the repair is auditable rather than assumed.

What the 150 figure is, and is not

limitation	what the record says
provider balance	96 of 150 eligible cases come from one provider (openai). A per-provider result from this set is not a comparison between providers.
category balance	the largest recorded category holds 58 of 150 cases and the smallest holds 1; 33 cases predate the field entirely. An unweighted score over this set is dominated by the largest category.
multi-hop	1 case. One observation cannot support any claim about multi-hop performance, and 150 does not change that.
protocol	<code>ACCEPTED_PROTOCOL_DEVIATION</code> — the preregistered pilot was not run
corpus reproduction	incomplete; 139 Anthropic documents and 2482 unbuildable identities outstanding
retrieval	<code>RETRIEVAL_BLOCKED</code>

33 of 150 cases predate the `reasoning_type` field entirely and carry no recorded category. That is reported rather than backfilled: assigning a category to a case years after its approval would be inventing review that did not happen.

Why the holdout is still not frozen

150 eligible cases can support the 30–40 / 70–100 split by size, so size is no longer the reason. What is missing is a split policy: the set is provider- and category-skewed and holds 1 genuine multi-hop case, so an unweighted split would decide by accident which categories the holdout can measure. Freezing also stays blocked while corpus reproduction is incomplete.

11b. Batch 007 — fixes shipped, pilot blocked

FIXES IMPLEMENTED AND VERIFIED — CORPUS RECOVERY EXHAUSTED AND FAILED — SAFEGUARDS ADDED — PILOT NOT RUN — STOPPED FOR INDEPENDENT REVIEW

The three defects batch 006's review recorded were implemented and verified against the real cases that exposed them. Defect **E**'s fix (`src/rag_v1/gold/factidentity.py`) detects the `GOLD-B005-11 / GOLD-B006-06` pair the owner caught by hand, on the shared triple `aws_bedrock_base_url → override → endpoint` , with 0 false positives inside batch 006. It flags rather than drops, because two libraries genuinely differing in behaviour is a real case and only a reviewer can tell the two apart.

WHY THE PILOT COULD NOT RUN

BLOCKED — the frozen evidence the pilot must draw from is not present in this environment

The pilot had to draw from the spans that failed batch 006 *only* because no builder could express them. Batch 006 recorded `removed.unbuildable = 2482` as an integer and discarded each fact's identity — **the set was counted, never persisted**. The corpus text needed to re-derive it lives only in Postgres, and that corpus is not present in the environment where the pilot was attempted.

This is the cost of a counter where a list was needed, and it is why the batch-006 generation report now records a corpus census rather than only a count.

Corpus reproduction was pursued and is **incomplete**: the OpenAI half is recovered and reproducible from pinned commits; 139 Anthropic documents as they stood on 2026-08-17 are outstanding, of which 13 are provably drifted and 125 are unverifiable without the frozen fingerprint. That work is recorded under CORPUS-001 and is not summarised further here.

The contract batch 007 was preregistered under

THE LINE

The authoring model may change **WORDING**. It may not change **MEANING**. It authors the **QUESTION**; it never invents the **FACT**.

Controlled evidence-grounded paraphrasing: where a deterministic template cannot express an explicit evidence fact, a model may author the question. An **authoring** change — the evidence stays frozen and exact, the ground truth is still read out of the source, and every existing gate still runs.

The order is the safeguard

1. frozen source evidence selected
2. literal source fact extracted
3. subject / relation / object recorded
4. atomic claims anchored
5. only then the natural question is paraphrased

Never: invent question → search for supporting evidence.

The entailment self-check — any failure drops the candidate

check	it fails when
A Does the exact evidence support the complete answer?	any part of the answer is not derivable from the anchored spans
B Does every atomic claim map to literal evidence?	a claim has no span whose text supports it
C Did the question introduce any new condition?	the question adds an 'if', 'when' or 'unless' the source lacks
D Did it broaden model/provider/platform scope?	the source names a model, provider, platform or surface and the question drops or generalises it

check	it fails when
E Did it reverse the relation?	the question's subject is the source's object and vice versa, and the relation is not symmetric
F Did it introduce a causal claim absent from the source?	the answer explains why, and the source only says what
G Could the answer be verified using only the exact evidence?	verifying it needs the surrounding document, a heading, or outside knowledge

Every existing gate still runs

gate	implemented in	behaviour
bare definition scope	<code>rag_v1.gold.scoping</code>	every span, independently
critical anaphora	<code>rag_v1.gold.anaphora</code>	blocks; noncritical flags
subject / relation direction	<code>rag_v1.gold.relations</code>	REVERSED and SUBJECT_MISMATCH both drop
question form matches evidence form	<code>rag_v1.gold.questionform</code>	negative-as-positive and truncated predicates drop
duplicate detection	<code>scripts/export_batch_007.py</code>	question text, span offsets, span text — and now the relation triple
critical strings	<code>rag_v1.gold.normalisation</code>	every one must be literally inside its own span
evidence hashes	<code>scripts/export_batch_007.py</code>	each span hashes to its own text
scope self-containment	<code>rag_v1.gold.scoping</code>	section_path is never claim scope
example-code restriction	<code>scripts/export_batch_007.py</code>	a sample configuration is not a rule
evidence size	<code>scripts/export_batch_007.py</code>	<500 preferred, 1000 soft cap, 1500 hard cap
provider / model scope	<code>scripts/export_batch_007.py</code>	a scoped source needs a scoped question
holdout eligibility	<code>rag_v1.gold.eligibility</code>	deterministic, run after owner approval

12. Tests and invariants

The suite is **941 passed**. Ten test files cover GOLD-001 specifically, including regression tests built from `GOLD-B005-01` , `-08` , `-10` , `-11` , `-15` and `-18` , and from batch 006's duplicate against `GOLD-B005-11` .

What the tests hold

- **Closed-batch immutability.** Every closure hash is recomputed from the records it covers. An edit to a closed batch fails the suite.
- **No retrieval leakage.** `retrieval_was_not_run` is asserted on every batch and every record, and each record is checked for retrieval-derived fields.
- **Frozen system hashes.** SYSTEM-A `9afcb5b7...` and SYSTEM-B `304c3509...` are compared against the frozen constants.
- **Evidence integrity.** Every span hashes to its own text; every critical string is literally inside its own span.
- **Taxonomy changes move no evidence.** Offsets, text and hashes are compared before and after every relabelling.
- **Nothing claims verification it lacks.** Generation artifacts must carry no decision; only the owner appears as a reviewer.
- **The paraphrasing contract.** Batch 007 cannot broaden scope, add conditions, reverse relations or introduce causal claims without a test failing.

Invariants, unchanged across all six batches

- No retrieval system has been run against any GOLD candidate at any point. `retrieval_was_not_run = true` ; `systems_executed = []` . No question was selected, ordered or worded because of retrieval difficulty.
- SYSTEM-A and SYSTEM-B remain frozen and unexecuted; the corpus snapshot, chunks, embeddings, fusion, DOC-C, top-k, routing, prompts and reranking are unchanged. All of this has been evaluation-authoring work only.
- No holdout is frozen and no validation split is frozen.
- Closed batches are never modified. Repairs live beside a generation artifact, never inside it; a new discovery about old tooling becomes an audit, an erratum or a future overlay, never a silent mutation.
- Claude's self-review is authoring, not verification. It has never been presented as independent verification, and no AI may set `human_verified` .

13. Commits and artifacts

commit	date	subject
fbb182c	2026-08-31	GOLD-001: one complete-record PDF for the whole evaluation
da0419c	2026-08-31	CORPUS-001: recovery report PDF
03df7d0	2026-08-31	CORPUS-001: the host is not reachable from here; two oracles, and the gap named exactly
84d0d6e	2026-08-31	CORPUS-001: recover the snapshot's two unknown parameters; corpus still not reproduced
6c6df94	2026-08-31	GOLD-001: admit HA-01..HA-60 from the packet of record and close at 150
3f5182d	2026-08-31	GOLD-001: record the HA packet split; admission still blocked on the master packet
600160e	2026-08-30	GOLD-001: OpenAI half recovered and certified; Anthropic half still missing
ca6524f	2026-08-30	GOLD-001: rehydration is certifiable in principle and refuted in practice
6bd75c3	2026-08-30	GOLD-001 batch 007 report: close the ChatGPT-project archive lead
22ed9cb	2026-08-30	GOLD-001: assess the 2026-08-17 results as recovery evidence; add a shape gate
7af7534	2026-08-30	GOLD-001 batch 007 report: record the host-side recovery evidence

commit	date	subject
278e1f4	2026-08-30	GOLD-001: corpus recovery exhausted; add the safeguards batch 006 lacked
6585a7e	2026-08-30	GOLD-001 batch 007 report: record the review recommendation and a recovery checklist
44a537c	2026-08-30	GOLD-001 batch 007: fixes E/F/G implemented; calibration pilot blocked

Where each figure in this document comes from

- experiments/GOLD-001/GOLD-001-eligibility-status.json — project totals and the per-batch eligibility figures.
- experiments/GOLD-001/GOLD-001-batch-00N-closure.json (N = 1...6) — acceptance, closure hashes, repairs, overrides, shortfalls, multi-hop results.
- evals/review/gold_review_batch_00N*.json — the candidate records themselves, and batch 001's v2 overlay.
- experiments/GOLD-001/b00N-review-decisions.json — the review findings and the generator defects each batch recorded.
- experiments/GOLD-001/GOLD-001-heading-parser-audit.json — the heading figures.
- experiments/GOLD-001/GOLD-001-batch-007-preregistration.json — the batch-007 contract.

Generated by scripts/build_gold001_complete_record.py. Every figure is read from the artifacts listed above at build time; none is typed into this document. The build refuses to run if any batch is missing a closure, if a closure's totals disagree with its records, if the eligibility gate re-run disagrees with any closure, if the project totals disagree with the sum of the batches, if any batch records a retrieval run, if a frozen system hash has moved, if a batch-007 artifact exists, or if the page would claim batch 007 reaches the project target. Raw provider documentation is not redistributed; the corpus is referenced by snapshot id and cases by character offset.